MINISTRY OF EDUCATION AND RESEARCH

Technical University of Moldova

Faculty of Computers, Informatics, and Microelectronics

Department of Software Engineering and Automation

MacOS Platform for Visual Environment for Real-Time Interface Prototyping for iOS Applications

Undergraduate Internship Report

Student:

_____

Dragomir Mindrescu, gr. FAF-221

Coordinator:

_____

Grade: _____

Chisinau, 2026

ABSTRACT

The thesis titled **MacOS Platform for Visual Environment for Real-Time Interface Prototyping for iOS Applications** addresses the gap between static design tools and native iOS prototyping workflows. The 30% milestone is dedicated primarily to rigorous analysis, requirements specification, system modeling, and an implementation chapter limited to the technologies used in the current prototype. The document is written in English and contains a title page, abstract, introduction, four main chapters, conclusions, and bibliography.

SwiftUI Builder is positioned as a no-code visual prototyping tool for macOS. Its currently validated capabilities include visual composition of interface blocks, contextual property editing, multi-device preview, JSON-based project persistence, and one-click deployment to the iOS Simulator through the companion `PreviewRunner` application. This combination supports early interface validation in a native runtime context and differentiates the project from static mockup tools and generic no-code platforms.

The report is organized around the official sequence required for the practice report. Chapter 1 performs the domain analysis by describing the current state of no-code and native iOS prototyping, the problem and stakeholder impact, competing solutions, the SWOT position of the proposed system, and the project's SMART objectives. Chapter 2 formalizes the requirements specification through actor identification, scope definition, and functional and non-functional requirements. Chapter 3 presents the system modeling artifacts through UML-based behavioral, structural, deployment, and persistence views. Chapter 4 focuses on system implementation at the current stage by justifying the technologies used and documenting the current engineering baseline.

The main contribution of the work at this stage is the alignment of the thesis narrative with the actual prototype and its archived codebase. The report treats the project honestly as a single-screen, file-based, simulator-backed prototyping environment while establishing a defensible foundation for the later implementation-heavy milestone.

Keywords: SwiftUI Builder, no-code prototyping, iOS Simulator, visual interface design, macOS

TABLE OF CONTENTS

LIST OF FIGURES . . . . . .

LIST OF TABLES . . . . . .

INTRODUCTION **1**

0.1 Problem Statement 1

0.2 Relevance and Objective 2

0.3 Methodology 2

0.4 Report Structure 2

1 DOMAIN ANALYSIS **4**

1.1 Current State of the Field in 2026 4

1.2 Problem and Stakeholder Impact 4

1.3 Competitive Analysis 5

1.4 SWOT Matrix 6

1.5 Target Actors and Persona 7

1.6 Solution Proposal 8

1.7 Validation Logic of the Proposed Direction 8

1.8 SMART Objectives 9

1.9 Expected Academic and Practical Contribution 10

2 REQUIREMENTS SPECIFICATION **11**

2.1 Actor Model and System Scope 11

2.2 Functional Requirements 12

2.3 Non-Functional Requirements 13

2.4 Constraints and Assumptions 14

2.5 Requirements Traceability 15

3 SYSTEM MODELING **16**

3.1 Modeling Strategy 16

3.2 Use Case Model 16

3.3 Activity Model of the Core Workflow 17

3.4 Sequence Model of Simulator Deployment 18

3.5 Class Model of the Core Types 18

3.6 Deployment Model 19

3.7 Persistence Model 20

3.8 High-Level Architecture Model 21
3.9 Modeling Synthesis 22
4 SYSTEM IMPLEMENTATION 23
4.1 Technology Selection Criteria 23
4.2 Technology Stack Overview 24
4.3 Native Application Layer: Swift and SwiftUI 24
4.4 Platform Integration Layer: AppKit and Foundation 25
4.5 Persistence Strategy: JSON and `Codable` 25
4.6 Runtime Validation Toolchain 25
4.7 Documentation and Modeling Tooling 26
4.8 Current Implementation Baseline 26
4.9 Repository and Codebase Organization 27
CONCLUSIONS 29
BIBLIOGRAPHY 31

LIST OF FIGURES

3.1 Use Case Diagram for the Primary Screen-Building Workflow 17
3.2 Activity View of the Core Editing and Validation Workflow 17
3.3 Sequence Diagram of the Simulator Deployment Flow 18
3.4 Class Diagram of the Core Shared Model 19
3.5 Deployment Diagram for the Current Local Validation Environment 20
3.6 Persistence Model of the JSON Project Structure 21
3.7 High-Level Structural View of SwiftUI Builder 22

LIST OF TABLES

1.1 Competitive Analysis of Relevant Tools 6
1.2 SWOT Matrix for SwiftUI Builder 7
1.3 SMART Objectives for the 30% Milestone 9

2.1 Actor Model for the 30% Milestone 11
2.2 Non-Functional Requirements for the 30% Milestone 14
2.3 Traceability from User Needs to Technical Responses 15

4.1 Technology Stack and Justification 24
4.2 Implemented Capability Baseline of the Archived Prototype 27
4.3 Representative Repository Milestones 27
4.4 Codebase Organization of the Archived Prototype 28

INTRODUCTION

By 2026, low-code and no-code development have moved from a niche productivity promise to a recognized direction in software engineering practice and research. Organizations increasingly expect visual development environments to shorten iteration loops, broaden participation in digital product creation, and reduce the translation loss between requirements, design artifacts, and implementation decisions [1], [2], [3], [4]. At the same time, native iOS development still relies on a comparatively technical toolchain centered on Swift, SwiftUI, Xcode, and platform-specific design conventions [5], [6], [7], [8]. This combination makes early interface validation faster in theory than in practice: ideas can be visualized quickly, but realistic native prototypes still often require programming knowledge and a full development environment.

The present report addresses this mismatch through **SwiftUI Builder**, developed under the subject **MacOS Platform for Visual Environment for Real-Time Interface Prototyping for iOS Applications**. The system combines a visual composition workspace, contextual property editing, device-aware preview, JSON-based project persistence, and automated execution in the iOS Simulator through the companion `PreviewRunner` target. At the 30% milestone, the emphasis still falls on the analytical and modeling foundation of the project, but the official report structure also requires a dedicated implementation chapter. At this stage, that chapter is interpreted strictly as a description and justification of the technologies used, not as a full feature-by-feature implementation narrative [9].

0.1 Problem Statement

The core problem is the persistent gap between design-centered tools and native validation workflows. Modern interface design tools are effective for layout exploration and stakeholder communication, but they remain mostly static or semi-interactive environments. They do not provide the same assurance as a prototype executed in a realistic iOS runtime context. On the opposite end, Xcode and SwiftUI provide the native path, but they assume technical skills, toolchain familiarity, and direct coding effort even for early exploratory work [10], [11].

This gap remains relevant in 2026 for three reasons. First, citizen-development and low-code initiatives continue to expand, which increases demand for tools that lower the entry barrier to software creation without discarding engineering discipline [3], [4]. Second, interface expectations in the Apple ecosystem remain high, which means that static mockups are often insufficient for evaluating layout fidelity, device-specific behavior, and visual polish [6], [8]. Third, early validation pressure is stronger than before: teams, individual developers, and startup-style product initiatives are expected to validate ideas earlier and cheaper, not after full implementation has already consumed substantial effort [12].

0.2 Relevance and Objective

The project is therefore motivated by the need for a local-first, macOS-native visual environment that can shorten the path from concept to realistic iOS prototype. The objective of the current milestone is not to claim that the entire native development process can be replaced. Instead, the milestone aims to establish a defensible project basis in which visual authoring, native preview, local persistence, and simulator-backed validation are treated as the core contribution of the prototype.

At this stage, the project is deliberately framed around verified capabilities:

- visual composition from reusable SwiftUI-oriented blocks;
- contextual property editing;
- preview on multiple Apple device presets and appearance modes;
- JSON project persistence;
- one-click execution in the iOS Simulator.

Direct SwiftUI code export remains a future direction rather than an implemented claim of the 30% milestone. This limitation is intentional. It keeps the thesis narrative aligned with the real prototype and prevents the report from overstating functionality that belongs to later stages.

0.3 Methodology

To establish the analytical and design basis of the thesis, the work combines several complementary methods:

- review of academic and technical literature on low-code development, prototyping, and native iOS tooling;
- analysis of archived project artifacts, including earlier proposal, presentation, and UML material;
- source-level inspection of the current SwiftUI Builder prototype and its shared model layer;
- formalization of functional and non-functional requirements;
- system modeling through UML and architecture-oriented views of the most important workflows and structures.

This methodology ensures that the report stays grounded in the implemented system while still providing a research-backed explanation of why the topic matters and how the architecture should evolve in later milestones.

0.4 Report Structure

The report is organized into four main chapters after the introduction. Chapter 1 presents the domain analysis by examining the current state of no-code and native iOS prototyping, the problem and stakeholder impact, competing solutions, the SWOT position of the proposed system, and the SMART objectives of

the project. Chapter 2 presents the requirements specification through actor identification, scope definition, and functional and non-functional requirements. Chapter 3 presents the system modeling artifacts through UML behavior views, structural views, deployment interpretation, and the persistence model. Chapter 4 presents the current implementation stage by justifying the selected technologies and documenting the current engineering baseline. The report ends with conclusions that summarize the present milestone and outline the next development-oriented stage.

1 DOMAIN ANALYSIS

1.1 Current State of the Field in 2026

The thesis topic is positioned at the intersection of low-code development, interface prototyping, and native Apple-platform tooling. The field is relevant because contemporary software creation increasingly values shorter validation loops, broader participation in product definition, and stronger alignment between requirements, design, and implementation. Academic work on low-code platforms shows that visual development environments are no longer treated only as simplified teaching tools; they are now discussed as a serious software-engineering phenomenon connected to productivity, user empowerment, model-driven practices, and citizen development [1], [2], [3], [4].

At the same time, the research literature also points out that low-code environments vary widely in maturity and scope. They often combine visual interface design, data modeling, and deployment support, but their technical assumptions differ substantially, and their promises may exceed their real engineering value if the domain is not analyzed carefully [1], [2]. This observation is important for the present thesis because it discourages a generic or marketing-driven interpretation of the topic. The value of the project must therefore be defined in relation to a precise gap, not merely through the broad language of no-code productivity.

For native iOS prototyping, the problem remains particularly visible in 2026. SwiftUI has reduced some of the boilerplate of interface development, but the native toolchain still expects familiarity with programming, Xcode configuration, and platform conventions [5], [6], [7]. In parallel, product teams increasingly need interface validation earlier in the lifecycle, when ideas are still volatile and expensive rework must be avoided. Prototyping remains valuable precisely because it helps clarify requirements, improve communication, and reduce ambiguity before full implementation begins [10], [11], [12].

1.2 Problem and Stakeholder Impact

The central problem addressed by the thesis is the lack of a lightweight workflow that connects visual editing with realistic native iOS validation. Design tools such as Figma and Sketch are efficient for static composition and review, but they do not provide a native iOS runtime environment as part of the same authoring loop [13], [14]. On the other hand, Xcode and SwiftUI support native execution, but they assume coding effort even when the immediate goal is only to validate interface structure, layout fidelity, or screen-level presentation [6], [7].

This gap affects several stakeholder groups:

– **Designers** need to validate interface decisions beyond static mockups, especially when screen pro-

portions, safe-area behavior, and runtime appearance matter.

- **Independent developers** need to iterate on screen concepts faster than manual SwiftUI authoring allows during the ideation phase.
- **Product stakeholders** need realistic artifacts that reduce ambiguity during reviews and early demonstrations.
- **Students and educators** benefit from a visual bridge between interface theory and native iOS implementation concepts.

The impact of the problem is not limited to convenience. Weak early validation increases the likelihood of rework, slows feedback cycles, and preserves a divide between design artifacts and implementation artifacts. The literature on prototyping and early-stage product validation supports the idea that prototypes can act as communication vehicles, specification surrogates, and requirement-refinement instruments when they are close enough to the eventual execution context [10], [11], [12]. For the present domain, this means that a native-facing prototype is more useful than a static board when the question is not only what a screen looks like, but whether it behaves and feels plausible as an iOS interface.

1.3 Competitive Analysis

The competitive scan covers four categories of relevant solutions: design-first environments, generic no-code builders, SwiftUI-oriented commercial products, and native development tools. The goal of the comparison is not to claim that these tools are universally inadequate. Instead, the comparison identifies which combination of features is still weakly supported: visual authoring with low entry barrier, native iOS orientation, and direct simulator-backed validation [7], [13], [15], [16], [17], [18], [19].

Table 1.1 - Competitive Analysis of Relevant Tools

Tool	Primary Orientation	Low-Code Entry	Native iOS Focus	Direct Simulator Use	Main Limitation for This Thesis Context
Figma / Sketch	Interface design and mockups	High	No	No	Strong for visual exploration, but the workflow ends before native runtime validation.
FlutterFlow	Cross-platform no-code builder	High	Partial	Indirect	Focuses on generated application projects rather than on a lightweight SwiftUI-centered prototyping loop.
Adalo / Bubble	Generic no-code application building	High	No	No	Prioritize web or platform-agnostic delivery, which reduces native iOS fidelity.
Judo / Builder.io	SwiftUI-oriented commercial tooling	Medium	Partial	Partial	Closer to the target domain, but commercial, opinionated, and not framed as a local-first academic prototype.
Xcode	Native development environment	Low	Yes	Yes	Offers full native control, but the entry barrier is too high for rapid visual ideation.

The analysis highlights a useful niche. Static design tools are strong in layout exploration, but weak in native execution. Xcode is strong in native execution, but weak as a low-friction visual prototyping environment. Generic no-code tools reduce the entry barrier, but often move away from native Apple-platform behavior. The proposed system is intended to occupy the space between these categories by remaining small in scope while emphasizing a very specific advantage: moving from visual editing to simulator-backed validation without abandoning the authoring workflow.

1.4 SWOT Matrix

The SWOT analysis summarizes the strategic position of the proposed system at the current milestone.

Table 1.2 - SWOT Matrix for SwiftUI Builder

Strengths	Weaknesses
– Native macOS implementation aligned with the Apple ecosystem. – Editing workflow built directly around SwiftUI-oriented blocks and Apple device presets. – Practical differentiator through one-click simulator deployment. – Local project ownership through JSON-based persistence.	– Current scope is limited to single-screen prototypes. – The component library is intentionally compact. – Direct SwiftUI code export is not part of the validated milestone core. – The prototype depends on macOS and Xcode command-line tooling.
Opportunities	**Threats**
– Ongoing adoption of low-code and citizen-development workflows. – Educational use as a bridge into native SwiftUI concepts. – Extension toward navigation modeling, richer blocks, and code export. – Increased demand for faster collaboration between design and implementation roles.	– Rapid SwiftUI evolution may force maintenance effort. – Larger commercial platforms may absorb adjacent use cases. – Narrow initial scope may reduce perceived usefulness if not extended carefully. – Apple ecosystem dependence limits portability to non-Apple environments.

The SWOT result supports the thesis direction. The project is not strongest when evaluated as a universal no-code platform, but it becomes more defensible when evaluated as a narrow, native-focused prototyping environment for the early validation stage.

1.5 Target Actors and Persona

The domain analysis identifies three primary user segments:

- independent iOS developers who need faster early-stage interface iteration;
- designers who collaborate with iOS teams and need more realistic prototypes than static boards;
- students and educators who benefit from a visual entry point into SwiftUI-oriented interface construction.

These segments share a common need: interface validation without the full cost of native implementation from the first step. Their expectations differ, but all of them benefit from a tool that reduces the distance between concept and a plausible iOS runtime artifact.

A representative persona is **Alex Rivera**, a 27-year-old independent iOS developer and design enthusiast. Alex wants to test an onboarding concept rapidly, verify the look of the screen on more than one Apple device size, and show the result to a stakeholder without creating a complete Xcode prototype manually. Alex's main pain points are repetitive SwiftUI layout coding, slow early iteration in Xcode, and the lack of a lightweight local tool that remains close to native iOS behavior.

1.6 Solution Proposal

The proposed solution is SwiftUI Builder, a macOS-native visual environment for real-time interface prototyping of iOS applications. The system does not attempt to replace the full software-development lifecycle. Instead, it focuses on a smaller and more realistic promise: enable users to compose a screen visually, configure its properties, preview it on Apple device presets, save it locally as a structured project, and validate it through a simulator-backed runtime path.

This proposal improves on existing alternatives in a targeted way:

- compared with design-first tools, it adds a direct path to native execution;
- compared with Xcode, it lowers the barrier for early layout experimentation;
- compared with generic no-code systems, it keeps the workflow centered on native iOS presentation rather than cross-platform abstraction.

The main value of the system is therefore not generic automation. Its value lies in narrowing a specific gap between static visual design and realistic native validation.

1.7 Validation Logic of the Proposed Direction

The proposed direction is deliberately narrow, and that narrowness is part of its validity. Three architectural choices explain why the solution is defensible for the 30% milestone.

First, the system is local-first rather than cloud-first. This choice is consistent with the current thesis objective because the core workflow depends on local device preview, local JSON persistence, and simulator execution on the same workstation. Introducing remote synchronization or browser-based authoring at this stage would broaden the scope without strengthening the main contribution. For the present milestone, the decisive question is not how to distribute prototypes across teams, but how to shorten the path between visual composition and native validation.

Second, the current implementation is intentionally limited to a single-screen model. This is not a weakness of planning, but a scope-control decision. A single-screen prototype is sufficient to validate the feasibility of the component model, the inspector interaction pattern, the preview layer, the persistence format, and the simulator-deployment chain. If these foundations are unstable, adding navigation and multi-screen orchestration would only increase complexity without improving the milestone's analytical quality.

Third, simulator-backed validation is prioritized over direct code export. Earlier project artifacts considered code generation as a differentiating direction, but the archived prototype provides stronger evidence for the simulator workflow than for production-ready export. The thesis therefore gains credibility by treating code export as future work and by treating runtime validation as the currently validated differentiator. This decision aligns the report with the real prototype and improves academic defensibility.

Taken together, these design choices show that the proposed solution is not a generic imitation of broader no-code platforms. It is a focused response to a precisely defined gap in the iOS prototyping workflow.

1.8 SMART Objectives

Based on the domain analysis, the project goal can be formulated as follows:

to provide a macOS-native visual environment for building and validating native iOS interface prototypes with immediate feedback and simulator-backed execution.

For the 30% milestone, this goal is decomposed into explicit SMART objectives shown in Table 1.3.

Table 1.3 - SMART Objectives for the 30% Milestone

ID	Objective	SMART Interpretation
SO1	Define the domain gap and compare the project with at least four relevant solution categories.	Specific to the field study, measurable through the competitive table and SWOT matrix, achievable within the milestone, relevant to the thesis justification, and completed by the 30% submission.
SO2	Formalize a system specification covering actors, scope, and numbered functional and non-functional requirements.	Specific to design preparation, measurable through the requirements chapter content, achievable with the available artifacts, relevant to later implementation, and due at the current milestone.
SO3	Document a validated prototype scope containing five reusable block types, three device presets, two appearance modes, JSON save/load support, and simulator deployment.	Specific and measurable through archived code and report evidence, achievable because these features already exist in the prototype, relevant to the thesis contribution, and time-bound to the current milestone baseline.
SO4	Model the most important system views through at least one use case, one activity, one sequence, one class/data model, one deployment view, and one persistence view.	Specific to system design, measurable through the selected UML diagrams, achievable using the current artifact base, relevant to architectural clarity, and completed as part of the 30% report.
SO5	Establish a repository-backed project foundation with documented commit history and code organization.	Specific to development initiation, measurable through repository evidence in the report, achievable with the current Git history and archived codebase, relevant to milestone transparency, and time-bound to the present submission.

The SMART decomposition is useful beyond formal compliance with the checklist. It also constrains the project narrative. If later implementation work diverges from these objectives, the thesis should be updated accordingly rather than defended through vague claims. In this sense, the SMART objectives function both as planning devices and as audit checkpoints for milestone honesty.

1.9 Expected Academic and Practical Contribution

The expected contribution of the thesis is twofold. From a practical perspective, the project aims to provide a focused prototyping workflow for a part of the iOS development lifecycle that is still underserved: early interface validation that is more realistic than static design boards, yet less expensive than manual Xcode prototyping from the first step. Even if the current scope remains intentionally limited, the system already illustrates how visual composition, local persistence, and simulator-backed execution can be combined into a coherent authoring loop.

From an academic perspective, the thesis contributes by documenting this workflow as a structured design problem rather than only as a software demo. The project is positioned against low-code research, prototyping literature, and native iOS constraints, and its architecture is described in a way that can be critiqued, extended, or compared in later work. This is especially relevant in a domain where many tools are described through marketing language rather than through explicit system models and milestone-bounded claims.

The project also has educational value. Because the prototype is built around SwiftUI-oriented blocks and Apple device presets, it may serve as an accessible bridge for students who understand interface goals conceptually but are not yet comfortable with the full native toolchain. That educational perspective does not replace the product rationale, but it strengthens the thesis relevance within the university context.

The domain analysis therefore confirms three conclusions. First, the thesis topic remains relevant in 2026 because the demand for low-friction validation continues to rise. Second, the proposed system is differentiated enough when evaluated against its precise niche. Third, the 30% milestone can be defended convincingly if it is framed as rigorous analysis and design preparation rather than as a premature implementation showcase.

2 REQUIREMENTS SPECIFICATION

This chapter translates the results of the domain analysis into a formal specification of what the system must do and under which quality constraints it must operate. At the 30% milestone, the specification is intentionally bounded to the validated scope of the current prototype rather than to a hypothetical full application builder. The chapter therefore formalizes the actor model, the system boundaries, the functional requirements, the non-functional requirements, and the traceability links that will later connect requirements to implementation and evaluation.

2.1 Actor Model and System Scope

The current system is intentionally simple from the actor perspective. It does not model multiple human roles such as administrator, guest, or cloud collaborator because the practice-report milestone targets a local, single-user prototyping environment. The actor model is nonetheless explicit because the report must define who interacts with the system, which external tools participate in the workflow, and where the system boundary ends.

Table 2.1 - Actor Model for the 30% Milestone

Actor / System	Category	Responsibility in the Current Milestone
User	Primary human actor	Creates, edits, saves, and validates an interface prototype in the macOS builder application.
Local File System	Supporting external system	Stores exported JSON project files and makes them available for reopening or simulator transfer.
`xcodebuild`	Supporting external tool	Builds the iOS runtime application used to validate the exported prototype.
`simctl` / iOS Simulator	Supporting external toolchain	Discovers, boots, installs, and launches the simulator target that executes the prototype.
`PreviewRunner`	Supporting runtime application	Loads the exported project and renders it using the shared data and view layer.

The scope of the current milestone includes the following validated capabilities:

- visual composition of a single prototype screen from reusable interface blocks;
- editing of textual, layout, and style properties through a contextual inspector;
- preview on multiple device presets and appearance modes;
- local save and reopen support through JSON project files;
- runtime validation by launching the prototype through `PreviewRunner` in the iOS Simulator.

The following capabilities are intentionally outside the scope of the current report stage:

- multi-screen flow authoring and navigation modeling;
- collaborative editing or cloud synchronization;

– direct export of production-ready SwiftUI source code;
– backend integration, business-logic composition, or database-backed application generation.

These boundaries are necessary for two reasons. First, they preserve implementation honesty and keep the report aligned with the real prototype. Second, they create a stable baseline for later milestone growth instead of mixing validated capabilities with aspirational roadmap items.

2.2 Functional Requirements

The functional requirements are grouped by the major user-visible responsibilities of the system. The identifiers defined in this section are reused later in the system-modeling and implementation chapters.

FR 1. Visual Composition

– **FR 1.1.** The system shall provide a component library containing reusable interface blocks.
– **FR 1.2.** The system shall support at least the following block types at the current milestone: Symbol, Headline, Body Copy, Primary Button, and List.
– **FR 1.3.** The system shall allow the user to add a new block to the canvas from the component library.
– **FR 1.4.** The system shall allow the user to duplicate, reset, select, and delete blocks already placed on the canvas.

FR 2. Property Editing

– **FR 2.1.** The system shall provide a contextual inspector for the currently selected block.
– **FR 2.2.** The inspector shall support editing of content, typography, spacing, padding, and color-related properties where applicable.
– **FR 2.3.** The inspector shall adapt its controls to the selected block type rather than exposing one fixed property set for all blocks.

FR 3. Preview and Device Configuration

– **FR 3.1.** The system shall provide a real-time preview of the currently edited screen.
– **FR 3.2.** The system shall support at least three device presets during the milestone: iPhone SE, iPhone 15 Pro, and iPad mini.
– **FR 3.3.** The system shall allow the user to switch between light and dark appearance modes.
– **FR 3.4.** The system shall allow zoom control of the preview workspace.

FR 4. Project Persistence

– **FR 4.1.** The system shall allow the user to save the current prototype as a JSON project file.
– **FR 4.2.** The saved project shall contain the project name, selected device preset, appearance mode, and the full list of placed blocks with their properties.
– **FR 4.3.** The system shall support reopening previously saved projects for continued editing or

previewing.

FR 5. Runtime Preview

- **FR 5.1.** The `PreviewRunner` application shall load the most recent exported project.
- **FR 5.2.** The runtime preview shall render blocks using the shared data model and shared rendering logic.
- **FR 5.3.** The runtime preview shall provide clear feedback when a valid project cannot be loaded.

FR 6. Simulator Deployment

- **FR 6.1.** The system shall support a one-click action for running the current prototype on the iOS Simulator.
- **FR 6.2.** The deployment sequence shall include project export, simulator discovery, build invocation, application installation, and runtime launch.
- **FR 6.3.** The system shall provide user-facing feedback when deployment fails at any stage.

Taken together, the functional requirements show that the current thesis stage is not limited to static visual editing. The system must support a complete local workflow that starts with composition and ends with native runtime validation.

2.3 Non-Functional Requirements

The non-functional requirements define the quality expectations and engineering constraints of the prototype. Because the current milestone remains a thesis-stage prototype, the targets are expressed as measurable design expectations rather than as production-level guarantees.

Table 2.2 - Non-Functional Requirements for the 30% Milestone

ID	Quality Attribute	Target for the Current Milestone
NFR 1	Performance and responsiveness	Preview updates, selection changes, and property edits should feel immediate during normal use; project save operations should complete in less than one second for a typical single-screen prototype of up to 50 blocks.
NFR 2	Usability and UI/UX quality	The interface should follow familiar macOS interaction patterns and Apple Human Interface Guidelines; only relevant controls should be shown for the selected block; a first-time user should be able to assemble a simple screen during a short guided session.
NFR 3	Reliability	Save operations should preserve project state in stable JSON form; invalid or missing project files should generate understandable errors rather than crashes; simulator failures should be surfaced explicitly.
NFR 4	Compatibility	The builder application should target modern macOS environments with Xcode command-line tools available; the runtime preview should operate in the iOS Simulator environment.
NFR 5	Security and privacy	The current milestone should remain local-first; project files should stay in the user's local environment; no cloud synchronization or remote data transfer should be required for the main workflow.
NFR 6	Maintainability and extensibility	The architecture should separate builder-specific logic, shared rendering logic, and runtime-preview logic; the data model should remain extendable with additional block types and later navigation capabilities.
NFR 7	Scalability within milestone scope	The system should remain understandable and usable for single-screen prototypes of moderate complexity, without claiming large-project or multi-user scalability at this stage.

The non-functional specification is especially important for the current subject because usability, local reliability, and technology alignment are part of the research value of the project. A macOS platform for visual prototyping would not be credible if it generated heavy friction, required hidden remote services, or behaved inconsistently between the editor and the runtime preview.

2.4 Constraints and Assumptions

The current system design operates under several explicit constraints and assumptions:

- the system is offline-first and stores project data locally rather than in a shared service;
- the current prototype models one screen at a time and does not yet represent complete application navigation;
- simulator deployment depends on Apple tooling such as `xcodebuild` and `simctl`, which means the workflow is intentionally bounded to the Apple ecosystem [7], [20];
- the project is evaluated as a prototyping and validation environment, not as a full application generator;

– future direct SwiftUI code export remains a roadmap item rather than a validated milestone capability.

These assumptions protect the report from over-claiming. The current practice report is strongest when it describes a narrow, validated system with clear boundaries rather than a broad platform with unclear evidence.

2.5 Requirements Traceability

The final step of the requirements chapter is to connect user needs to the technical responses that will be modeled in the next chapter and justified technologically in the implementation chapter.

Table 2.3 - Traceability from User Needs to Technical Responses

User Need	Primary Requirement Groups	Expected Result
Rapid visual experimentation	FR 1, FR 2, FR 3	The user can compose, edit, and review a screen without entering manual SwiftUI code.
Realistic native validation	FR 3, FR 5, FR 6	The prototype can be previewed on Apple device presets and executed through the iOS Simulator.
Reliable local project persistence	FR 4, NFR 3, NFR 5	Work can be stored and reopened locally in a structured exchange format without relying on remote services.
Low learning barrier for non-specialists	Actor Model, FR 2, NFR 2	Property editing remains contextual and visually guided instead of toolchain-heavy.
Stable base for later milestones	NFR 6, NFR 7	The current milestone establishes a maintainable foundation for later navigation, richer blocks, and possible code-export experiments.

The requirements specification therefore establishes a clear contract for the remaining chapters. The next chapter explains how these requirements are modeled formally, while the implementation chapter justifies why the selected technologies are appropriate for satisfying them within the scope of the present milestone.

3 SYSTEM MODELING

This chapter presents the formal models used to describe the behavior, structure, deployment context, and persistence logic of SwiftUI Builder. The purpose of the chapter is not to maximize the number of diagrams, but to explain the most important system views in a readable and academically defensible form. The selected models were chosen because they correspond directly to the validated workflow of the current prototype: visual composition, contextual editing, local persistence, and simulator-backed validation.

3.1 Modeling Strategy

The modeling set follows UML-oriented conventions and keeps only the views that materially improve understanding of the present milestone [21]. This decision is important for two reasons. First, a practice report should not overload the reader with large but weakly interpreted diagrams. Second, the current prototype is still intentionally narrow, so the most valuable models are those that explain the core flow in a precise way: what the user does, how the system responds over time, which core types participate, and how the local execution environment is organized.

The modeling chapter is therefore organized into three groups:

- behavioral models that describe the principal interaction flow;
- structural models that explain the main system entities and their relationships;
- deployment and persistence models that show where the system runs and how project state is represented.

3.2 Use Case Model

The use case model identifies the most important interaction expected from the current milestone. The main actor is the user working in the macOS builder application, while the local file system, Apple toolchain, and simulator infrastructure act as supporting external systems. The goal is not only to place interface blocks visually, but to complete a closed workflow that includes editing, previewing, saving, and native runtime validation.

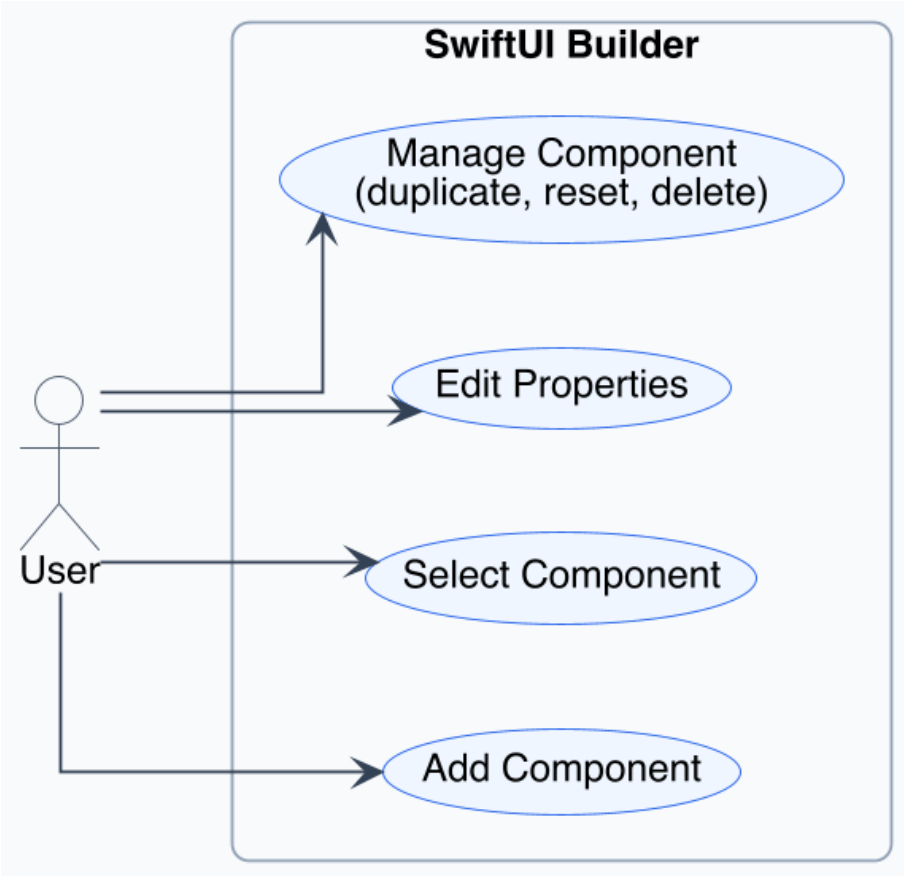

Figure 3.1 - Use Case Diagram for the Primary Screen-Building Workflow

Figure 3.1 shows that the current system promise is broader than a static design canvas. The user can add and manage blocks, inspect the resulting screen, persist the project locally, and trigger runtime validation in the simulator. In requirements terms, the use case aggregates the main concerns defined earlier in FR 1 through FR 6. In thesis terms, it demonstrates that the project targets a realistic design-to-validation loop rather than a generic mockup editor.

3.3 Activity Model of the Core Workflow

The activity model expands the main use case into an operational flow. It starts with entering the builder workspace, continues through an iterative editing loop, and branches toward either local saving or simulator execution. This model is important because the system is meant to support repeated experimentation rather than a one-shot export process.

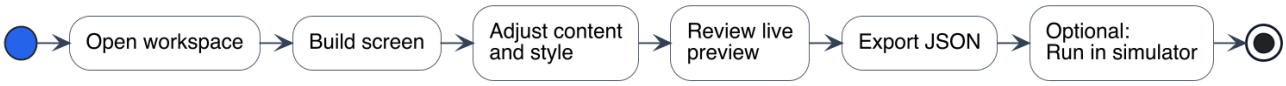

Figure 3.2 - Activity View of the Core Editing and Validation Workflow

The activity in Figure 3.2 emphasizes three ideas. First, the editing process is iterative and user-

driven. Second, preview is not a separate later step, but a continuous feedback mechanism embedded in the workflow. Third, saving and simulator execution are modeled as deliberate decisions rather than hidden background behavior. This makes the diagram particularly useful for presentation purposes because it explains the end-to-end thesis workflow in a compact form.

3.4 Sequence Model of Simulator Deployment

The sequence model captures the most distinctive behavior of the project: simulator-backed validation initiated directly from the builder. A static mockup tool usually ends its responsibility at export time. In contrast, SwiftUI Builder extends the workflow into project serialization, build orchestration, simulator preparation, installation, and application launch.

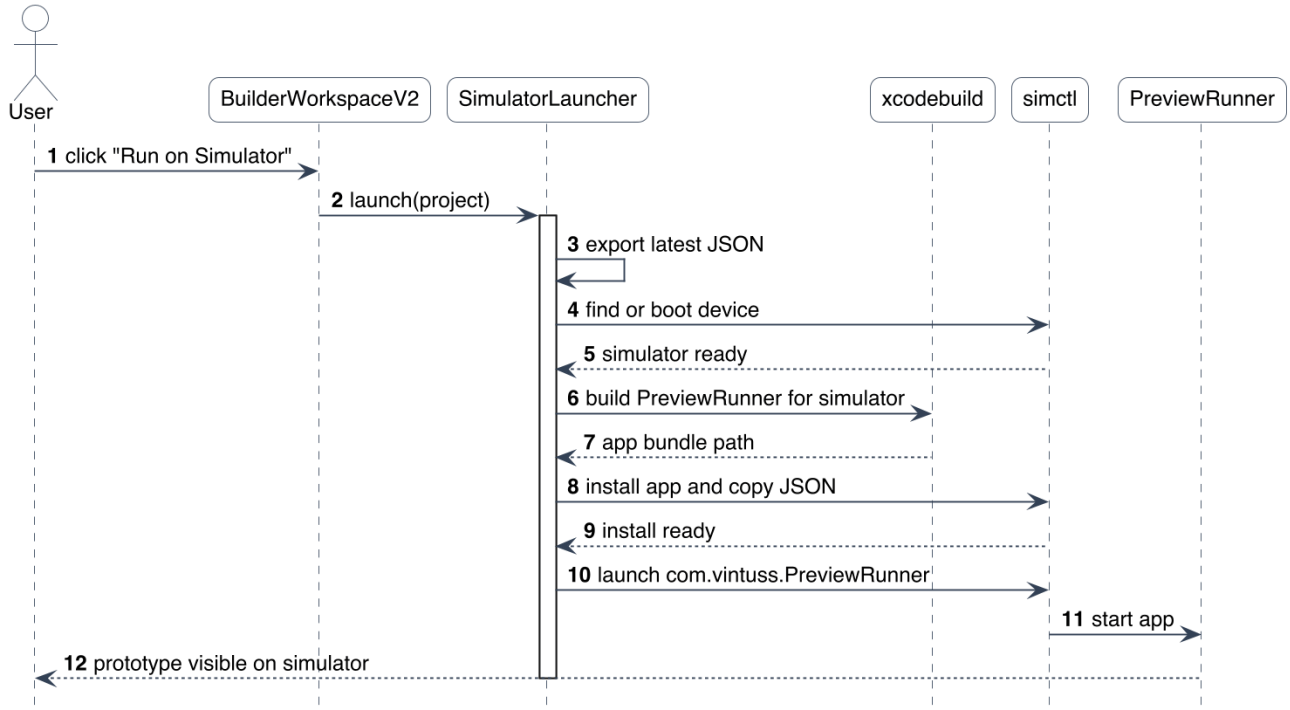

Figure 3.3 - Sequence Diagram of the Simulator Deployment Flow

Figure 3.3 formalizes how the user-facing workspace collaborates with `SimulatorLauncher`, the Apple command-line toolchain, and the `PreviewRunner` application. The value of this diagram is analytical as much as technical. It makes visible the exact orchestration chain that differentiates the thesis topic from a visual editor that stops at static export. It also justifies why error reporting and local toolchain assumptions must be handled explicitly in the implementation chapter.

3.5 Class Model of the Core Types

The class model focuses on the minimum set of types required to understand the current architecture. The diagram deliberately excludes many minor supporting types in order to preserve readability. For the

present milestone, the core analytical value comes from seeing how editable blocks, project serialization, workspace coordination, export logic, and simulator launching fit together.

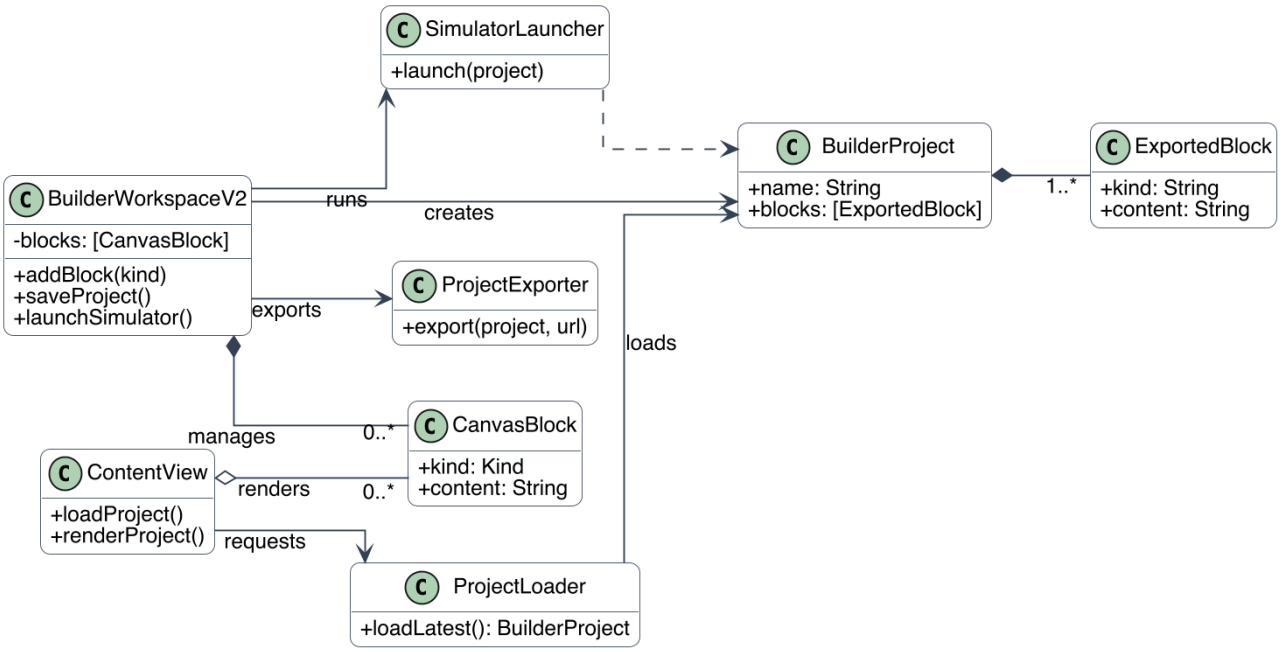

Figure 3.4 - Class Diagram of the Core Shared Model

The class view in Figure 3.4 should be read as a conceptual model rather than as a line-by-line reverse engineering artifact. It highlights who owns state, how the screen description is persisted, and where the transition from visual editing to runtime validation is initiated. This level of abstraction is appropriate for the current report because it supports technical reasoning without overwhelming the reader with low-level implementation details.

3.6 Deployment Model

The deployment model explains where the system runs physically and which nodes participate in the current validation workflow. Because the project is local-first and milestone-bounded, the deployment view is intentionally simple: one macOS workstation, local storage, Apple build and simulator tooling, and an iOS Simulator runtime.

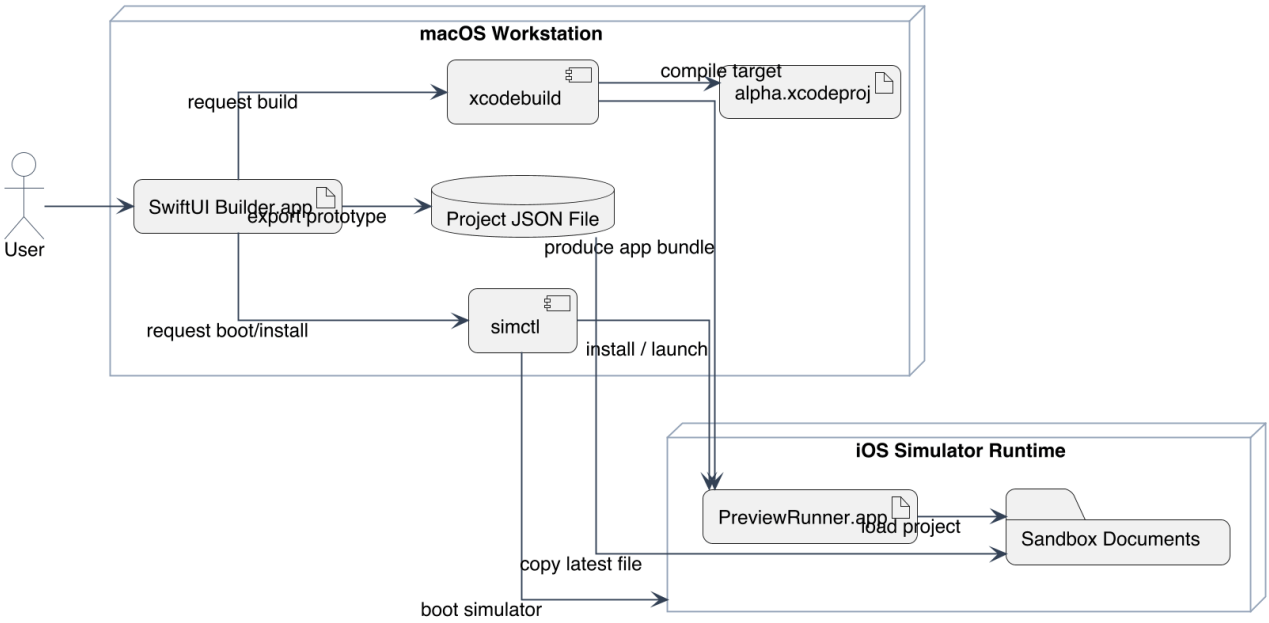

Figure 3.5 - Deployment Diagram for the Current Local Validation Environment

The deployment diagram is important for the audit because it shows that the physical environment is coherent with the project claim. No backend server, cloud runtime, or external database is implied. The builder application, the local JSON export, and the runtime validation target all remain on the same workstation, which matches both the educational scope of the milestone and the technical reality of the current prototype.

3.7 Persistence Model

The current system does not use a relational database, so a classical entity-relationship diagram would misrepresent the implemented design. The correct model for this milestone is the persistence structure of the JSON document exchanged between the builder and the runtime side.

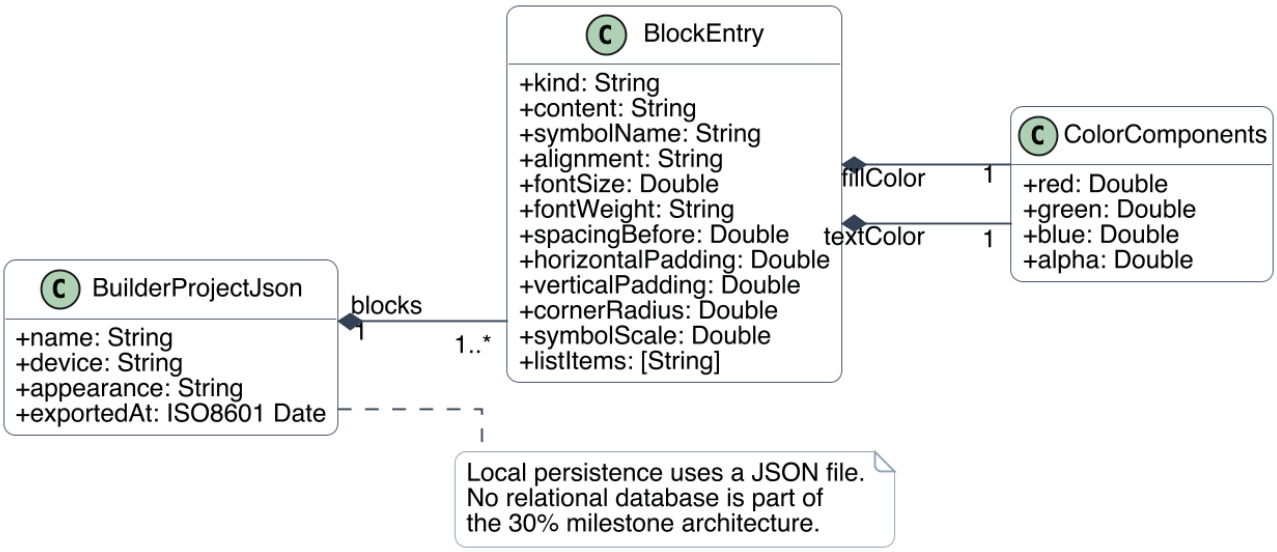

Figure 3.6 - Persistence Model of the JSON Project Structure

Figure 3.6 shows that the root object stores project-level metadata such as project name, selected device preset, appearance mode, and export time, while each block entry stores the content and styling data required for runtime reconstruction. This model is analytically valuable because it documents the real transfer boundary of the system. The persistence schema is the contract that allows the visual builder and the runtime preview to remain separated while still using the same project description.

3.8 High-Level Architecture Model

At the architectural level, the system is organized as a builder-runner pair connected through a local exchange format. The builder application owns authoring and orchestration responsibilities, while the runtime application owns loading and rendering responsibilities.

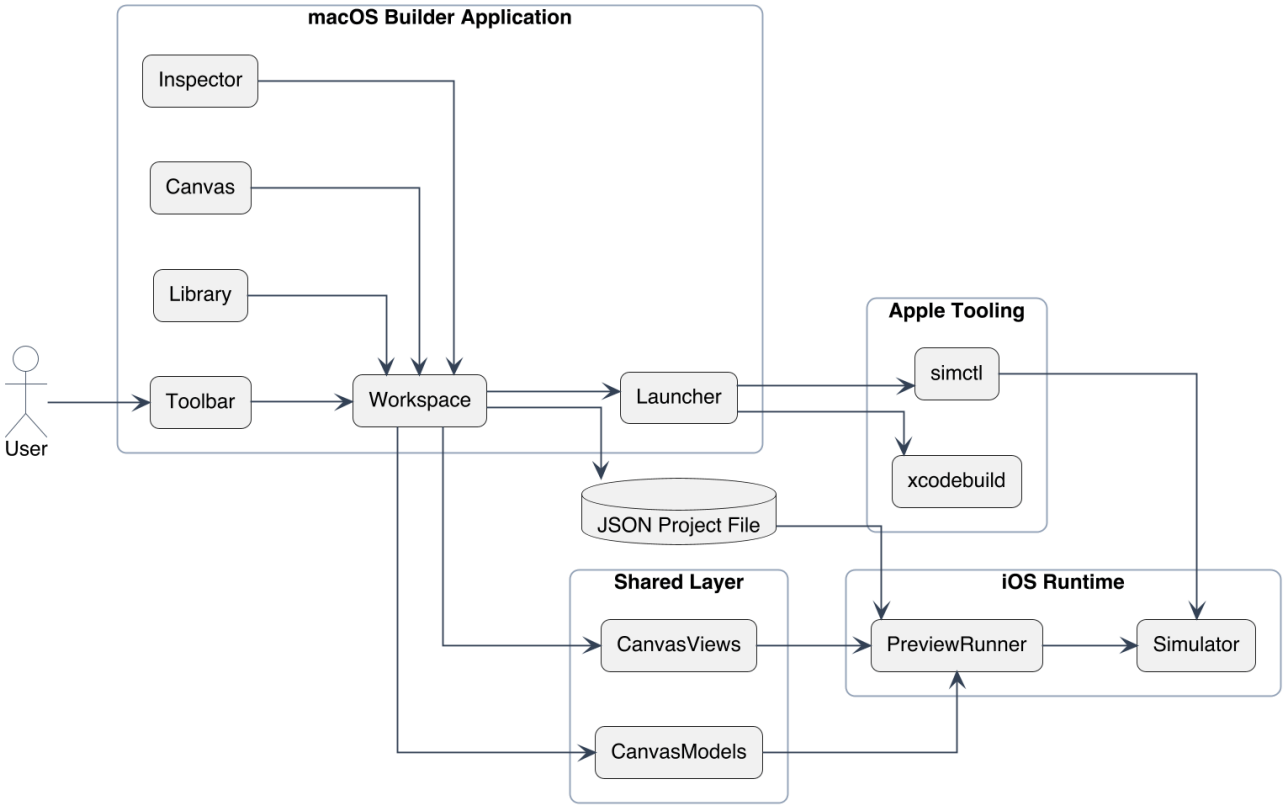

Figure 3.7 - High-Level Structural View of SwiftUI Builder

The architecture in Figure 3.7 clarifies why the project should not be interpreted as a monolithic desktop editor. The builder, shared layer, runtime target, and Apple toolchain each serve a distinct role. This separation supports maintainability, makes the JSON persistence boundary explicit, and prepares the project for later extensions such as richer component sets or multi-screen support.

3.9 Modeling Synthesis

Taken together, the diagrams in this chapter provide a coherent model set for the 30% milestone. The use case model defines the user goal, the activity model explains the editing loop, the sequence model formalizes simulator-backed validation, the class model identifies the core conceptual types, the deployment model bounds the physical environment, the persistence model documents the storage contract, and the architecture model connects the entire system into one readable structural view.

This synthesis is important for the practice report because the diagrams are not included merely to satisfy a formal requirement. They establish that the project has a well-defined internal logic and that the later implementation work can proceed from an explicit design basis rather than from ad-hoc feature growth.

4 SYSTEM IMPLEMENTATION

At the current milestone, the implementation chapter is interpreted strictly as a description and justification of the technologies used in the project. The objective is not to provide a full walkthrough of every feature, but to explain why the chosen stack is appropriate for a macOS platform dedicated to real-time interface prototyping for iOS applications. The discussion therefore focuses on alignment with the thesis scope, engineering consequences, and the current implementation baseline already present in the archived prototype.

4.1 Technology Selection Criteria

The implementation stack was selected according to five criteria:

- direct alignment with the Apple ecosystem targeted by the thesis subject;
- support for a local-first workflow without mandatory cloud infrastructure;
- low translation loss between design-time representation and runtime iOS rendering;
- maintainability at thesis scale;
- compatibility with the current milestone, where the emphasis is on prototyping and validation rather than on full application generation.

These criteria explain why the project did not adopt a generic web stack such as React with a Node.js backend, nor a database-centered architecture, nor a cross-platform no-code platform strategy. Those alternatives can be powerful in other contexts, but they would introduce unnecessary architectural distance between the builder environment and the native iOS runtime that this thesis explicitly tries to approximate.

4.2 Technology Stack Overview

Table 4.1 - Technology Stack and Justification

Layer	Technology	Justification
Builder application	Swift + SwiftUI	Enables a native macOS interface and keeps the editing environment close to the target iOS presentation technology.
macOS integration	AppKit + Foundation	Supports file-system access, process orchestration, and platform-specific behavior needed for export and simulator control.
Runtime preview	SwiftUI in `PreviewRunner`	Preserves visual consistency between the builder-side representation and the runtime-side rendering path.
Persistence	JSON encoded with `Codable`	Provides a lightweight, inspectable, and extensible project-exchange format appropriate for a local prototype workflow [22].
Build and launch automation	`xcodebuild` and `simctl`	Reuses the standard Apple toolchain instead of introducing a custom deployment mechanism; this is consistent with the goal of native validation [7], [20].
Modeling notation	UML 2.5.1 + PlantUML	Supports precise but maintainable description of behavior, structure, deployment, and persistence views [21].
Documentation workflow	LaTeX + BibLaTeX + GitHub	Supports versioned academic writing, citation management, and reproducible report builds.

The stack overview is useful, but it becomes meaningful only when each selection is interpreted in the context of the thesis subject. For that reason, the following sections justify the main layers individually.

4.3 Native Application Layer: Swift and SwiftUI

Swift was selected because the project targets Apple platforms directly and aims to reduce the gap between visual authoring and native runtime validation. Using Swift as the implementation language avoids the need for an intermediate translation layer between the design environment and the target rendering environment [5]. This is particularly important for the thesis because the research contribution is tied to fidelity of prototyping rather than to broad cross-platform reach.

SwiftUI is the natural interface technology in this context. It allows the builder-side preview and the runtime-side preview to rely on the same general rendering paradigm, which improves consistency between authoring and validation [6]. If the builder had been implemented as a web application or through a generic canvas toolkit, the visual representation would have been farther from the behavior and styling conventions of the final iOS runtime. That alternative would weaken the central thesis claim that the platform supports realistic interface prototyping rather than merely interface sketching.

There is also a maintainability benefit. SwiftUI encourages declarative interface construction, which fits the builder metaphor of assembling a screen from reusable blocks and parameterized properties. Even though the current prototype does not export complete SwiftUI source code, it already benefits from a shared mental model between visual composition and runtime rendering.

4.4 Platform Integration Layer: AppKit and Foundation

Pure SwiftUI is not sufficient for all the tasks required by the current milestone. The project needs local file handling, process invocation, and tighter interaction with the macOS environment. For these reasons, AppKit and Foundation remain necessary supporting technologies.

Foundation is used because the current workflow depends on structured serialization, filesystem access, process management, and basic data handling. These responsibilities are not peripheral. They are essential for saving projects, reopening them, preparing the exported payload for the runtime target, and coordinating simulator execution.

AppKit is relevant because a desktop authoring tool benefits from platform-specific behavior that goes beyond the basic mobile-style interaction model. The thesis subject is not a generic cross-platform builder but a macOS platform, so the implementation is justified in using macOS-native integration points where the workflow requires them. This choice increases technical alignment with the subject instead of diluting it.

4.5 Persistence Strategy: JSON and `Codable`

The current project uses JSON as the exchange and persistence format. This choice is appropriate because the system is local-first, milestone-bounded, and single-user at the present stage. JSON is lightweight, human-inspectable, and easy to evolve for a prototype whose main data structure is a screen composed of blocks and their properties [22].

The use of `Codable` strengthens this decision. It reduces manual serialization complexity and keeps the persistence model close to the in-memory model representation. This is important for reliability and maintainability because the persistence schema is not a secondary export convenience. It is the real contract between the builder and the `PreviewRunner` application.

Equally important is what was not chosen. A relational database would add unnecessary infrastructural complexity and would misrepresent the architecture of the system at this stage. The prototype does not need concurrent access control, server-hosted persistence, or relational querying. It needs a stable local project document that can be written quickly, inspected easily, and transferred directly into the runtime validation workflow.

4.6 Runtime Validation Toolchain

One of the most distinctive technical aspects of the project is the decision to validate prototypes through the Apple toolchain rather than through a simulated in-editor preview alone. This is why `xcodebuild`, `simctl`, and the dedicated `PreviewRunner` target are central to the implementation story.

`xcodebuild` is used because it provides the standard build interface for Apple-platform projects.

Using the official tool instead of a custom build abstraction preserves compatibility with the real development environment and reduces the risk of a prototype-specific execution path that would diverge from native behavior [7].

`simctl` is used because the prototype needs repeatable interaction with the iOS Simulator: device discovery, boot control, application installation, and launch [20]. This choice is academically important because it turns the thesis from a simple visual editor into a platform that can bridge visual design and runtime validation.

The `PreviewRunner` application completes the toolchain. It acts as a controlled runtime target whose responsibility is to load the exported project and render it with the shared models and views. This separation is technically cleaner than embedding all behavior into one monolithic application, and it supports the thesis claim that the system is structured around a builder-runner workflow.

4.7 Documentation and Modeling Tooling

The implementation story of the project also includes the tools used to keep the development process disciplined. GitHub is justified because the milestone requires repository evidence and meaningful commits. More importantly, version control is the mechanism through which report evolution, diagram updates, and implementation changes remain traceable rather than ad-hoc.

PlantUML is justified because the project requires several UML diagrams that need to evolve with the codebase. A text-based diagram workflow is particularly appropriate in a thesis repository because it supports versioned edits, consistent regeneration, and easier maintenance compared with manually redrawing every update.

LaTeX and BibLaTeX are justified because the report itself is a structured engineering deliverable. They support reproducible compilation, precise reference management, consistent chaptering, and better long-document control than an ad-hoc office-document workflow. This is especially relevant now that the report must satisfy strict formatting, sequence, and bibliography requirements.

4.8 Current Implementation Baseline

Although the present milestone is documentation-heavy, the report is supported by a real implementation archive rather than by diagrams alone. A direct source inspection of the archived prototype confirms approximately 2,466 lines of Swift code organized around the builder application, the shared model and rendering layer, and the `PreviewRunner` runtime target.

Table 4.2 - Implemented Capability Baseline of the Archived Prototype

Capability	Current State	Implementation Evidence
Reusable block set	Implemented	Five block types are represented in the shared model and exposed through the component library.
Contextual property editing	Implemented	The inspector adapts controls to the currently selected block and supports typography, spacing, color, and layout changes.
Device-aware preview	Implemented	The builder supports three Apple device presets and two appearance modes in the preview workflow.
Local project persistence	Implemented	The builder exports structured JSON through `Codable`-based models and reuses the same schema on the runtime side.
Simulator-backed validation	Implemented	The launcher orchestrates build, simulator discovery, installation, and application launch through Apple command-line tooling.
Multi-screen authoring	Not yet implemented	This remains outside the validated scope of the 30% milestone.
Direct SwiftUI code export	Not yet implemented	This remains a planned extension and is not claimed as current functionality.

This baseline is important because it prevents the implementation chapter from becoming abstract justification disconnected from reality. The selected technologies are not only theoretically suitable; they are already reflected in the current project artifacts.

4.9 Repository and Codebase Organization

The milestone also requires proof that the project has been initiated and managed as an engineering artifact. The current repository is hosted on GitHub at `https://github.com/VintusS/Thesis` and contains meaningful commits that demonstrate staged evolution from template setup to thesis-specific modeling and content work.

Table 4.3 - Representative Repository Milestones

Commit	Role in Evolution	Interpretation for the Milestone Audit
`f46ad54`	Initial commit	Establishes repository origin and baseline configuration.
`76b9095`	Template clone	Introduces the report workspace and initial project scaffolding.
`54ae428`	Build fix	Shows environment stabilization and reproducible LaTeX compilation work.
`c4f9061`	First iteration	Marks the transition from template state to thesis-specific content.
`99b7c64`	UML diagrams	Demonstrates structured modeling work and documentation maturation.

The codebase itself also exhibits a coherent module structure rather than an ad-hoc collection of files.

Table 4.4 - Codebase Organization of the Archived Prototype

Module or File Group	Responsibility
`Entry layer`	Application entry and root view composition through the app definition and the top-level content view.
`Workspace coordinator`	Central state owner for blocks, selection, project name, device, appearance, export, and simulator launch actions.
`Builder UI modules`	User-interface modules responsible for composition, preview, property editing, and top-level actions.
`Shared model/rendering layer`	Shared model and rendering layer reused between the builder and runtime targets.
`Simulator automation`	Encapsulation of build, simulator discovery, installation, and launch orchestration.

The implementation chapter therefore shows that the current milestone is already backed by an appropriate technical base. The technology choices are consistent with the thesis subject, the project scope, and the validated prototype artifacts. This creates a defensible basis for the next milestone, where the emphasis can move from technology justification toward richer implementation evidence and evaluation scenarios.

CONCLUSIONS

The 30% milestone establishes a coherent analytical and design foundation for the SwiftUI Builder thesis. The report now documents the project as a macOS-native visual prototyping environment for iOS interfaces, centered on capabilities that are already supported by the current project artifacts and codebase. This positioning is important because it replaces earlier ambiguity around direct code export with a more accurate and defensible thesis narrative based on visual editing, device-aware preview, local persistence, and simulator-backed validation.

The main achievements of the current stage can be summarized as follows:

- the current 2026 field context was analyzed with reference to low-code, prototyping, and native iOS constraints;
- the project gap between static design tools and native validation was clarified through competitive analysis and SWOT positioning;
- the technical material was reorganized into separate chapters for requirements specification, system modeling, and system implementation, which aligns the report with the current practice-report rules;
- the documented storage and deployment models were aligned with the implemented file-based prototype rather than with an artificial database-centered narrative.

The current prototype is already substantial enough to support the thesis direction, but several limitations remain:

- the system currently models one screen at a time rather than full navigation flows;
- the component library is intentionally compact and does not yet cover a broader set of iOS interface patterns;
- the workflow is optimized for runtime validation rather than for direct SwiftUI code generation;
- the tool remains dependent on macOS and the Apple development ecosystem.

These limitations define the logical direction of the next milestones. In the near term, the requirements, models, and technology narrative must be kept consistent with the evolving codebase, and the implementation should be extended carefully rather than broadened only at the documentation level. The 70% milestone should then shift the center of gravity toward implementation evidence, validation scenarios, and possibly richer navigation or component support. The final thesis should consolidate the full contribution, discuss limitations more critically, and revisit future-facing capabilities such as SwiftUI code export only if they are implemented convincingly by that stage.

In conclusion, the present milestone demonstrates that SwiftUI Builder is a viable and academically relevant thesis topic. The project already supports a meaningful design-time and validation-time workflow,

while the current report structure provides a stable basis for the implementation and evaluation chapters that will follow.

BIBLIOGRAPHY

[1] A. C. Bock and U. Frank, "Low-code platform," *Business & Information Systems Engineering*, vol. 63, no. 6, pp. 733–740, 2021. DOI: 10.1007/s12599-021-00726-8

[2] D. D. Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, "Low-code development and model-driven engineering: Two sides of the same coin?" *Software and Systems Modeling*, vol. 21, no. 2, pp. 437–446, 2022. DOI: 10.1007/s10270-021-00970-2

[3] A. Viljoen, A. Hein, and H. Krcmar, "Low-code development platform ecosystems," *Electronic Markets*, vol. 36, p. 12, 2026. DOI: 10.1007/s12525-025-00848-x

[4] B. Binzer and T. J. Winkler, "Democratizing software development: A systematic multivocal literature review and research agenda on citizen development," in *Proceedings of the 13th International Conference on Software Business*, ser. Lecture Notes in Business Information Processing, vol. 463, 2022, pp. 244–259. DOI: 10.1007/978-3-031-20706-8_17

[5] Apple Inc. "The swift programming language," Accessed: Mar. 23, 2026. [Online]. Available: https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[6] Apple Inc. "Swiftui documentation," Accessed: Mar. 23, 2026. [Online]. Available: https://developer.apple.com/documentation/swiftui

[7] Apple Inc. "Xcode documentation," Accessed: Mar. 23, 2026. [Online]. Available: https://developer.apple.com/documentation/xcode

[8] Apple Inc. "Human interface guidelines," Accessed: Mar. 23, 2026. [Online]. Available: https://developer.apple.com/design/human-interface-guidelines

[9] Technical University of Moldova. "Ghid: Elaborarea si sustinerea proiectelor/tezelor de licenta," Accessed: Mar. 23, 2026. [Online]. Available: https://utm.md/wp-content/uploads/2019/12/Ghid-Elaborarea-si-sustinerea-proiectelor-tezelor-de-licenta.pdf

[10] S. Ogata and S. Matsuura, "A review method for UML requirements analysis model employing system-side prototyping," *SpringerPlus*, vol. 2, p. 134, 2013. DOI: 10.1186/2193-1801-2-134

[11] S. D. Nelson, G. D. Fiol, H. Hanseler, B. I. Crouch, and M. R. Cummins, "Software prototyping: A case report of refining user requirements for a health information exchange dashboard," *Applied Clinical Informatics*, vol. 7, no. 1, pp. 22–32, 2016. DOI: 10.4338/ACI-2015-07-CR-0091

[12] K.-K. Kemell, A. Nguyen-Duc, M. Suoranta, and P. Abrahamsson, "Startcards — a method for early-stage software startups," *Information and Software Technology*, vol. 160, p. 107 224, 2023. DOI: 10.1016/j.infsof.2023.107224

[13] Figma, Inc. "Figma," Accessed: Mar. 23, 2026. [Online]. Available: https://www.figma.com/

[14] Sketch B.V. "Sketch," Accessed: Mar. 23, 2026. [Online]. Available: https://www.sketch.com/

[15] FlutterFlow, Inc. "Flutterflow," Accessed: Mar. 23, 2026. [Online]. Available: https://flutterflow.io/

[16] Adalo. "Adalo," Accessed: Mar. 23, 2026. [Online]. Available: https://www.adalo.com/

[17] Bubble Group, Inc. "Bubble," Accessed: Mar. 23, 2026. [Online]. Available: https://bubble.io/

[18] Judo. "Judo for swiftui," Accessed: Mar. 23, 2026. [Online]. Available: https://www.judo.app/

[19] Builder.io. "Builder.io documentation for swiftui," Accessed: Mar. 23, 2026. [Online]. Available: https://www.builder.io/c/docs/swiftui

[20] Apple Inc. "Running your app in simulator or on a device," Accessed: Mar. 23, 2026. [Online]. Available: https://developer.apple.com/documentation/xcode/running-your-app-in-simulator-or-on-a-device

[21] Object Management Group, *Unified modeling language specification, version 2.5.1*, 2021. Accessed: Mar. 23, 2026. [Online]. Available: https://www.omg.org/spec/UML/2.5.1/

[22] ECMA International. "Ecma-404: The json data interchange syntax," Accessed: Mar. 23, 2026. [Online]. Available: https://ecma-international.org/publications-and-standards/standards/ecma-404/